

Práctica 3: Búsqueda con Adversarios (Minimax)

1. Introducción

En las prácticas anteriores, el agente estaba solo. Ahora, el entorno es **hostil**. Tienes un adversario (Agente B) cuyo único objetivo es ganar, lo que implica que tú pierdas.

En esta práctica implementaremos un **Agente Basado en Minimax** para competir en el juego de Tron. A diferencia de la búsqueda A*, donde buscábamos el camino más corto, aquí buscamos el **movimiento óptimo asumiendo que el rival jugará de forma perfecta para perjudicarnos**.

El escenario se basa en una variante del juego "Tron" dentro de un laberinto. Dos agentes compiten por atravesar el laberinto desde extremos opuestos. La dificultad radica en que el rastro que deja cada agente se convierte en una pared, limitando el espacio de movimiento progresivamente.

Se parte de un laberinto rectangular.

- **Jugador A (Agente):** Comienza en la Posición de entrada ("E") Su objetivo es llegar a la columna derecha (Este) o sobrevivir si el oponente choca.
- **Jugador B (Agente Oponente):** Comienza en la Salida ("S") Su objetivo es llegar a la columna izquierda (Oeste) o sobrevivir si el oponente choca o se queda sin movimientos.

SI el jugador devuelve null o choca perderá igualmente. Podemos decidir el comportamiento del agente en este caso porque no cambia el resultado, sólo la salida por pantalla del último movimiento.

- Un jugador **pierde** inmediatamente si:
 - Choca contra una pared del laberinto (#).
 - Choca contra el rastro dejado por el oponente.
 - Choca contra su propio rastro.
 - No tiene movimientos legales disponibles.
- Si ambos chocan cabeza con cabeza en el mismo turno, es un empate.
- Si no tienen movimientos disponibles devuelven *null* o una acción por defecto.

Los agentes van dejando un rastro (a para el Jugador A, b para el Jugador B).

```

Laberinto Estado Actual:
  A avanza hacia el este, B hacia el oeste

#####
#aaaaaa#  #  Salida de A, aquí estaba la "E"
##### Bbbb # <-- Posición actual de B
##aa#  # ##b #
# a      # b #
# a#      b##
# a#  ##  # b##
#####A # b # <-- Posición actual de A
#      ##  bb# Salida de B, aquí estaba la "S"
#####

Jugador A  decide mover: S
Jugador B  decide mover: O
  
```

2. Arquitectura del Código:

Para que os centréis exclusivamente en la Inteligencia Artificial, se os proporciona un **Motor de Juego completo**. (Código Base - No tocar)

El archivo `TronP3Base.java` incluye:

1. **entorno:** Maneja la física.
 - Detecta colisiones (Muros #, Estelas a/b).
 - Gestiona los turnos.
 - **Simulación:** Permite hacer un movimiento y deshacerlo. Esto es vital para que vuestro agente pueda "imaginar" futuros sin alterar el juego real.

En el código, si ejecutamos `moverAgente('N')`, el agente pinta su muro en el array `grid[][]`. Luego entra en árbol recursivo (simulando los turnos del oponente). Cuando el algoritmo recursivo finalmente devuelve una puntuación final y regresa hacia arriba, el tablero se ha quedado sucio con todos esos movimientos simulados. Por lo tanto, justo en la línea de código estrictamente posterior al cálculo del Minimax recursivo, hay que obligar a llamar a `deshacerMovimiento()`.

```

Función Algoritmo( entornoActual, profundidadRestante, jugadorActual ):

  [Paso 1: ¿Es el final del camino?]
  SI la partida acabó o la profundidadRestante es 0:
    Llamar a mi función de Evaluación Heurística inventada (la de los pesos).
    Devolver la puntuación.

  [Paso 2: Preparación]
  Crear una variable que guarde 'laPuntuacionMasExtrema', iniciada de forma opuesta a mis intereses
  (Si soy yo, un número hiper negativo. Si es mi rival, hiper positivo).

  [Paso 3: Bucle de posibilidades]
  POR CADA ACCIÓN en [Norte, Sur, Este, Oeste]:
    SI puedo 'moverAgente' en esa cámara del tablero:

      1. Muevo la pieza en el tablero compartido.
      2. Evalúo el futuro llamando recursivamente a 'AlgoritmoAvanzado'
         (pasándole la profundidadRestante - 1, y cambiándole el turno al rival).
      3. Llamo a 'deshacerMovimiento' para que el tablero compartido regrese a su estado original
      4. ¿Es la puntuación del futuro de este turno mejor que 'laPuntuacionMasExtrema' que recordaba?
         Si lo es, machaco el valor y lo guardo como el mejor movimiento de momento.

  [Paso 4: Veredicto]
  Devolver al turno superior 'laPuntuacionMasExtrema'.
  
```

2. **Clase Tron_P3_Base (main):** Alterna turnos entre el Jugador A y el Jugador B y renderiza el tablero.

4. Tareas a realizar

1. **Agente Minimax (Profundidad Limitada):** Implementar un agente que utilice el algoritmo Minimax con una profundidad de búsqueda determinada. Dado que el árbol de juego es demasiado grande para explorarlo completo, se debe cortar la búsqueda y evaluar con una función de utilidad si no hemos llegado al final y se ha alcanzado la profundidad máxima.
2. **Función de Evaluación Heurística:** se debe diseñar una función heurística $h(n)$ que estime la utilidad del estado. Se sugiere considerar alguna de las siguientes, combinaciones de ellas o una propia. :
 - Distancia Manhattan a la meta (columna opuesta). (ojo cuanto más cerca deber ser mayor para MAX y menor para MIN al contrario de lo que expresa tal cual la distancia a la columna).
 - Número de movimientos libres disponibles en una dirección (movilidad). Añade información sobre que movimientos dan mejor probabilidades.
3. **Implementación de Poda Alfa-Beta:** Optimizar el algoritmo Minimax para poder aumentar la profundidad de búsqueda sin penalizar el tiempo de cómputo excesivamente.
4. **Simulación:** Ejecutar distintas partidas entre:
 - Agente Minimax vs Agente Aleatorio.
 - Agente Minimax vs Agente Alfa beta (con diferentes heurísticas o profundidades).

5. Estructura del Informe

1. **Diseño de la Heurística:** Explicar detalladamente la fórmula matemática utilizada para evaluar los estados intermedios. Qué se pretende y pruebas de funcionamiento.
2. **Análisis de Profundidad:** Comparar memoria usada e "inteligencia" percibida (capacidad de ganar) del agente a profundidad 2, 4 y 6 en cada algoritmo.
3. **Comparativa Alfa-Beta:** Gráfico o Tabla mostrando la reducción de nodos visitados al activar la poda Alfa-Beta frente al Minimax estándar para esas tres profundidades. El gráfico puede ser hecho con cualquier herramienta o a mano.

6. Rúbrica de Evaluación

- **Suspense:** El agente Minimax no funciona o juega aleatoriamente. El informe de la heurística es insuficiente.
- **Aprobado:** El agente Minimax bloquea correctamente jugadas suicidas inmediatas y utiliza una heurística básica.
- **Notable:** Implementación de Poda Alfa-Beta. La heurística incluye conceptos avanzados (control de territorio o distancia al rival). Comparativa Alfa-Beta de los nodos respecto de minimax
- **Sobresaliente:** El agente es capaz de ganar consistentemente al agente aleatorio y presenta batalla contra sí mismo. Se comparan distintas heurísticas. El código es modular y simple.

Entrega el 15 de Mayo, defensa en las semanas siguientes

Anexo: El concepto de "Deshacer Movimiento" en Búsqueda Adversarial

Cuando utilizamos algoritmos como **Minimax** o **Alfa-Beta**, estamos construyendo un árbol mental de todos los posibles futuros del juego. Para explorar estos futuros, necesitamos simular los movimientos.

Imaginad que tenemos un único tablero físico en la memoria del ordenador (nuestra clase `Entorno`) y no podemos hacer una fotocopia del tablero cada vez que pensamos en un movimiento (porque consumiría demasiada memoria y tiempo). Por tanto, la regla principal que sigue el algoritmo es: **"Toco la pieza para ver qué pasa, y luego la devuelvo a su sitio"**.

1. Entendiendo "Deshacer Movimiento" con una Ejecución Real

Imaginad un mapa muy pequeño con nuestro agente **A** (turno de Max) en la esquina superior izquierda (1,1) y el enemigo **B** (turno de Min) en la inferior derecha (3,3).

Vamos a ver cómo explora el algoritmo un árbol de decisiones a **profundidad 2** (nosotros simulamos un movimiento y luego simulamos cómo respondería el rival).

Este es nuestro tablero físico, la única instancia de la clase `Entorno` que existe en el programa:

```
--- Inicio de la Traza ---
#####
#A   #
#    #
#   B#
#####
```

Turno 1: Turno de A (Simulamos la primera opción: Sur)

El algoritmo comienza evaluando la primera acción válida para A: **Mover al Sur ('S')**.

```
ROOT: Iniciando evaluación para A
-> [ROOT] A PRUEBA mover al S (hacia 2,1)
```

Al llamar a `moverAgente('A', "S")`, el agente 'A' se desplaza a la casilla (2,1) y deja su rastro (una 'a' minúscula) en (1,1). Como compartimos el mismo tablero para toda la simulación, el mapa físico de la memoria cambia y se ve así:

```
#####
#a   #
#A   #
#   B#
#####
```

Turno 2: Turno de B (Responder al movimiento de A)

El turno de Max le delega el control a Min, pasándole **este mapa modificado**. Ahora Min debe evaluar sus opciones de respuesta. Prueba primero a moverse al **Norte ('N')**:

```
[Profundidad: 1] Turno de B (esMax: false) | Alfa: -INF Beta: INF
-> B PRUEBA mover al N (hacia 2,3)
```

Fijaos cómo el tablero vuelve a mutar, añadiendo el movimiento de B (dejando rastro 'b'):

```
#####
#a  #
#A  B#
#   b#
#####
[Hoja] Devuelve evaluación del tablero: 0
```

La profundidad de búsqueda llega a cero (nodo hoja) y la función evalúa el tablero devolviendo un valor de 0.

Aquí se usa :B (Min) no ha terminado su turno. Tiene que probar dentro de su bucle la siguiente opción: **Mover al Oeste ('O')**. ¿Qué pasaría si mueve al Oeste sin deshacer su movimiento anterior al Norte? ¡Que el agente B se movería desde su posición ficticia (2,3) dejando otro rastro, destrozando por completo el tablero!

Por eso, **justo después** de volver de la llamada recursiva anterior, el código ejecuta el deshacer:

```
<- DESHACE movimiento N de B
```

el tablero "retrocede en el tiempo". El rastro 'b' desaparece y el agente **B** vuelve a (3,3):

```
#####
#a  #
#A  #
#   B#
#####
```

Y solo entonces, el agente B está listo para simular su siguiente movimiento (Oeste) partiendo del punto de origen correcto:

```
-> B PRUEBA mover al O (hacia 3,2)
#####
#a  #
#A  #
#  Bb#
#####
[Hoja] Devuelve evaluación del tablero: -1
<- DESHACE movimiento O de B
```

Vuelta a la Raíz: Max prueba su segunda opción (Este)

Una vez que el rival (Min) ha terminado de probar todas sus respuestas posibles, devuelve el control a la raíz (al bucle inicial de A).

Ahora le toca a **A** evaluar si era mejor moverse al **Este ('E')** en el turno inicial. Pero, si miráis el mapa, 'A' sigue estando en la casilla Sur (2,1) dejando el rastro 'a'. Tenemos que deshacer también el movimiento inicial de Max.

```
<- [ROOT] DESHACE movimiento S de A
```

El tablero físico *vuelve* a su estado inicial original:

```
#####
#A  #
#   #
#   B#
#####
```

Y ahora sí, el agente **A** puede probar de forma segura la rama del Este. El proceso de simulación comienza de nuevo sobre un mapa limpio:

```
-> [ROOT] A PRUEBA mover al E (hacia 1,2)
#####
#aA #
#   #
#   B#
#####
[Profundidad: 1] Turno de B (esMax: false) | Alfa: -INF Beta: INF
-> B PRUEBA mover al N ... (y la búsqueda continúa)
```

2. Pseudocódigo: Poda Alfa-Beta Normativa

Aquí podéis ver cómo se integra esta lógica de "hacer y deshacer" en el algoritmo clásico, separando el turno de Max y el turno de Min. Fijaos en las 3 fases comentadas en el interior de los bucles: **Hacer**, **Delegar** y **Deshacer**.

Función para el turno de MAX (nuestro agente)

Busca **maximizar** la puntuación.

```
Función valorMax(mapa, profundidad, alfa, beta):
    Si profundidad == 0 o el juego ha terminado:
        return evaluar(mapa) // Devuelve la puntuación del tablero

    mejorValor = -INFINITO

    Para cada accion en ["N", "S", "E", "O"]:
        Si mapa.moverAgente(accion):
            // 1. HACER el movimiento (modifica el mapa en memoria)

            // 2. DELEGAR el turno al rival desde este nuevo estado
            valor = valorMin(mapa, profundidad - 1, alfa, beta)

            // 3. DESHACER el movimiento (restaura el mapa al estado original)
            mapa.deshacerMovimiento(accion)

            // 4. Actualizar nuestro mejor valor encontrado
            Si valor > mejorValor:
                mejorValor = valor

            // 5. Actualizar Alfa (lo mejor que Max ya tiene asegurado)
            Si mejorValor > alfa:
                alfa = mejorValor

            // 6. Poda Beta
            // Si nuestro valor ya es mejor que lo que el rival (Min) está dispuesto
            // a permitirnos (beta), el rival nunca nos dejará llegar a esta rama.
            Si mejorValor >= beta:
                romper bucle // Dejamos de explorar las siguientes acciones

    return mejorValor
```

Función para el turno de MIN (el rival)

Busca **minimizar** la puntuación.

```
Función valorMin(mapa, profundidad, alfa, beta):
    Si profundidad == 0 o el juego ha terminado:
        return evaluar(mapa)

    mejorValor = +INFINITO

    Para cada accion en ["N", "S", "E", "O"]:
        Si mapa.moverAgenteRival(accion):
            // 1. HACER el movimiento del rival

            // 2. DELEGAR el turno de vuelta a nosotros
            valor = valorMax(mapa, profundidad - 1, alfa, beta)

            // 3. DESHACER el movimiento del rival
            mapa.deshacerMovimientoRival(accion)

            // 4. Actualizar el mejor valor (buscando lo menor posible)
            Si valor < mejorValor:
                mejorValor = valor

            // 5. Actualizar Beta (lo peor que Min nos puede forzar a aceptar)
            Si mejorValor < beta:
                beta = mejorValor

            // 6. Poda Alfa
            // Si el valor que el rival ha encontrado es peor (menor) que lo que
            // Max ya tiene asegurado por otro camino (alfa), Max nunca elegirá esto.
            Si mejorValor <= alfa:
                romper bucle // El rival deja de explorar

    return mejorValor
```

3. Resumen

Como podéis observar en las trazas y en la vida de nuestro entorno de juego:

1. **Simulamos:** Movemos modificando el mapa compartido en memoria.
2. **Delegamos:** Llamamos recursivamente a la función contraria pasando ese mapa alterado.
3. **Restauramos:** Deshacemos el movimiento **antes** de continuar con la siguiente vuelta del bucle `for` para probar la siguiente acción de la lista.

Si omitiéramos la línea de `deshacerMovimiento`, los agentes acabarían teletransportándose o duplicando sus rastros durante la simulación de futuros hipotéticos, y la inteligencia artificial evaluaría estados que son completamente imposibles de dar en una partida real.